

Appunti del corso di **Basi di Di Dati**

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis

Matteo Drago

Indice

1	Introduzione	1
1.1	Perchè studiare i PL?	1
1.2	Cosa sono i PL?	2
1.3	Cosa significa programmare?	2
1.4	Contesti di sviluppo	3
1.5	Classificazione dei PL	3
1.6	Implementazione di un PL	4
1.7	Realizzare una macchina astratta	5
1.7.1	Soluzione interpretativa	7
1.7.1.1	Operazioni	7
1.7.1.2	Struttura	8
1.7.2	Soluzione compilativa	8
1.7.2.1	Struttura	9
1.7.3	Soluzione ibrida	9
2	Descrivere i linguaggi di programmazione	11
2.1	Sintassi	11
2.1.1	Descrivere la sintassi	12
2.1.2	Grammatiche CF	12
2.1.2.1	Notazione BNF	14
2.1.3	Descrivere un semplice linguaggio imperativo	14
2.1.3.1	Analisi semantica	15
2.2	Semantica	15
2.3	Categorie sintattiche	15
3	Comandi	16

Introduzione

Il problema principale dell'informatica consiste nel voler far eseguire ad una macchina algoritmi che manipolano dati.

macchina $\xrightarrow{\text{esegue}}$ algoritmo $\xrightarrow{\text{manipolazione}}$ dati

Il primo meccanismo che viene associato alla storia dei computer è la **macchina di Babbage**, che può essere visto come il primo "computer" perchè nasce per eseguire calcoli matematici complessi in modo automatico.

Sucessivamente, c'è un'evoluzione e il computer viene riconosciuto come macchina programmabile (dare delle istruzioni alla macchina per ottenere un determinato risultato).

In questo contesto nasce **Enigma**, uno strumento per decodificare un codice mediante un algoritmo di forza bruta che tenta tutte le combinazioni.

La prima architettura che darà origine all'architettura dei computer moderni è l'**architettura di Von Neumann**, un'architettura hardware che comprende solo il linguaggio binario, un linguaggio costituito esclusivamente da stringhe di bit (0 e 1) e che può essere utilizzato per codificare qualunque altro linguaggio, ma non è comprensibile dall'essere umano.

Questo fa sì che l'uomo per poter interagire e programmare macchine, necessita di un'evoluzione dei linguaggi, ha bisogno di costruire un linguaggio comprensibile in modo più immediato.

1.1 Perchè studiare i PL?

Le motivazioni sono diverse:

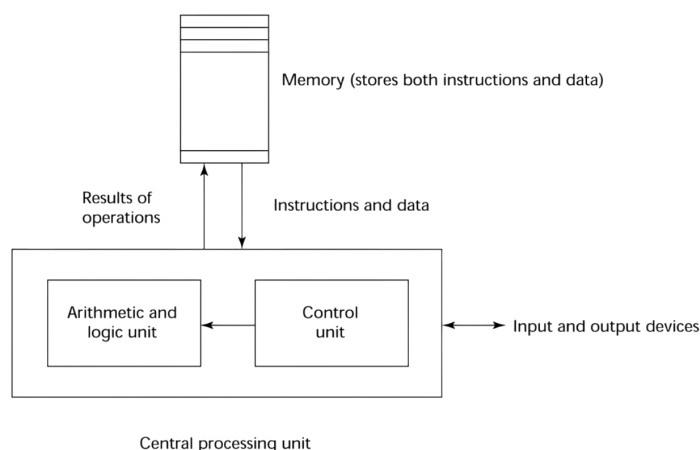
- ▷ migliorare la capacità di esprimere idee in forma algoritmica
- ▷ scegliere appropriatamente un linguaggio
- ▷ migliorare la capacità di studiare o progettare nuovi linguaggi o costrutti
- ▷ comprendere meglio il significato dell'implementazione dei linguaggi di programmazione

1.2 Cosa sono i PL?

La nascita e l'evoluzione dei linguaggi di programmazione è, anche, dovuto, all'architettura della macchina che bisogna programmare.

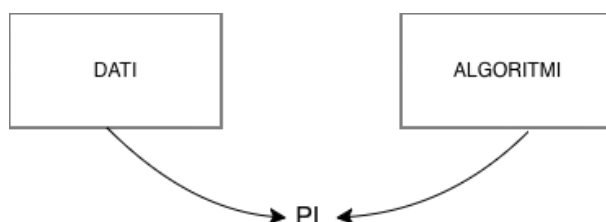
I computer moderni nascono dall'architettura di Von Neumann.

Questa è una tipologia di architettura hardware con programma memorizzato, dove i dati e istruzioni condividono la stessa area di memoria.



1.3 Cosa significa programmare?

Il linguaggio di programmazione deve essere uno strumento che permetta di descrivere algoritmi e rappresentare i dati.



Il linguaggio binario è un linguaggio di programmazione in quanto permette di rappresentare algoritmi e dati, ma non è umanamente utilizzabile o interpretabile, in quanto potrebbe essere usata come dato o un dato interpretato come istruzione.

Ciò significa, che un linguaggio di programmazione deve permettere di superare queste difficoltà, fornendo una chiara distinzione tra dato e istruzione.

Definizione: algoritmo

Un **algoritmo** è una sequenza infinita di passi di computazione, ovvero scompone un calcolo complesso in passi elementari.

Un algoritmo è un concetto astratto che trova forma concreta in un programma (sequenza finita di istruzioni).

Definizione: dati

I **dati** sono informazioni memorizzate concretamente in celle di memoria e astratte in elementi che il linguaggio può manipolare (variabili).

Definizione: programma

Un **programma** è la concretizzazione di un algoritmo che manipola dati.

Definizione: linguaggio di programmazione

Un **linguaggio di programmazione** è una notazione per scrivere un programma, ovvero serve a dare forma concreta agli algoritmi.

Il programma, quindi, costituisce la **sintassi**.

Un programma è necessariamente un oggetto finito (sempre concreto), mentre il calcolo di una funzione richiede l'esecuzione di un insieme di passi primitivi, che, però possono essere infiniti.

Ciò significa che un programma è una rappresentazione finita di un insieme, potenzialmente infinito, di passi primitivi di computazione.

Nota

I linguaggi di programmazione devono dare specifica finita di oggetti potenzialmente infiniti.

1.4 Contesti di sviluppo

Il contesto in cui il linguaggio deve operare determina le funzionalità che deve gestire efficacemente:

- ▷ **applicazioni scientifiche** (es. Fortran)
- ▷ **applicazioni economiche** (es. COBOL)
- ▷ **applicazioni artificiali** (es. LISP)
- ▷ **programmazione di sistemi** (es. C)
- ▷ **web software** (es. HTML (markup), PHP (scripting), ecc.)

1.5 Classificazione dei PL

I linguaggi di programmazione si possono classificare in:

- ◇ **basso livello**: linguaggi con caratteristiche specifiche dipendenti dall'architettura che si sta programmando (es. linguaggio binario, Assembly)
- ◇ **alto livello**: linguaggi che permettono una programmazione strutturata in cui dati e istruzioni hanno rappresentazioni diverse

I linguaggi ad alto livello si possono classificare in base a:

- **paradigma di programmazione**
 - linguaggi procedurali: codice diviso in subrutine che eseguono compiti semplici basati su salti condizionali (goto) [es. COBOL]
 - linguaggi strutturati (imperativi): procedurale senza goto (es. C, Pascal, ADA, ec..)
 - linguaggi orientati agli oggetti: classe elemento centrale della programmazione. Un linguaggio deve avere tre proprietà:
 - * incapsulamento
 - * ereditarietà
 - * polimorfismo
 - linguaggi dichiarativi: basati su componenti della matematica
 - * logici: basati sulla logica proposizionale del primo ordine, in cui si effettuano calcoli tramite dimostrazione di fatti (es. PROLOG)
 - * matematici: basati sul concetto di applicazione di funzione (es. LISP, ML)
- **termini generazionali**
 - I° generazione:

↓
 - II° generazione:

1.6 Implementazione di un PL

Implementare un linguaggio significa eseguire un programma nel linguaggio di programmazione in una macchina.

Sicuramente, il modo in cui deve essere implementato è direttamente collegato all'architettura che deve programmare, ma anche al paradigma (metodologie) che il linguaggio di programmazione utilizza per costruire programmi.

Implementare un linguaggio significa realizzare la macchina (astratta) che interpreta il linguaggio.

Nota

La macchina è astratta perchè, per rappresentare la macchina fisica (concreta), utilizza un linguaggio che è più astratto del linguaggio binario.

Definizione: macchina astratta

Dato un linguaggio L di programmazione, la **macchina astratta** M_L per L , è un insieme di strutture dati e algoritmi (è un programma) che permettono di memorizzare ed eseguire i programmi scritti in L .

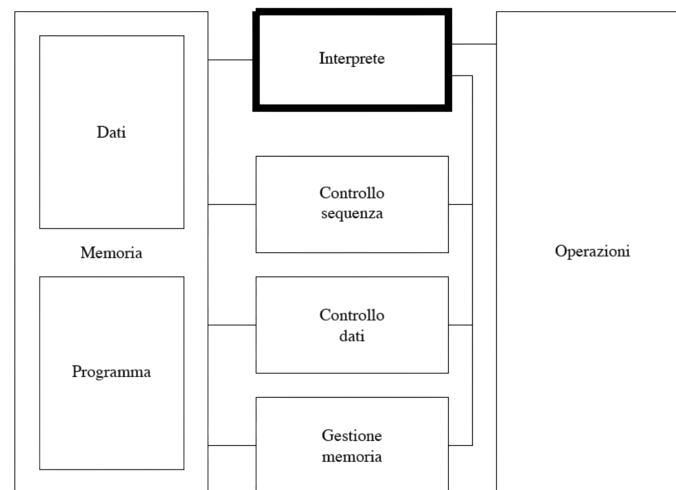


Figura 1.1: struttura della macchina astratta

Il linguaggio L , riconosciuto (interpretato) dalla macchina astratta M_L , viene anche chiamato **linguaggio macchina**.

Definizione: linguaggio macchina

Il **linguaggio macchina** è l'insieme di tutte le stringhe interpretabili da M .

1.7 Realizzare una macchina astratta

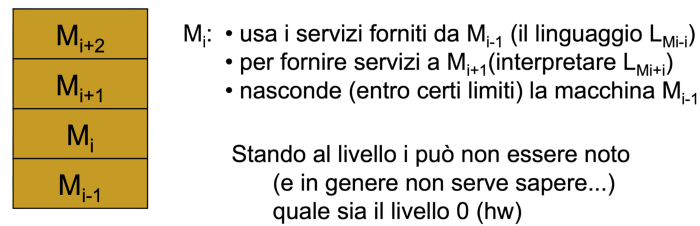
Una qualsiasi macchina astratta per L , per essere eseguita, deve utilizzare qualche dispositivo fisico che concretamente esegue le istruzioni di L .

Questo significa che in qualunque sistema informatico esiste sempre una macchina HW, ma non significa che tutte le macchine sono realizzate a livello HW, anzi per poter controllare la complessità di un sistema informatico, la realizzazione della macchina astratta può avvenire a vari livelli:

- ▷ **HW:** macchina realizzata in forma embedded e il linguaggio macchina è il linguaggio fisico/binario
 - massima velocità
 - nessuna flessibilità
- ▷ **FW:** macchina realizzata mediante simulazione/emulazione, ovvero strutture dati e algoritmi sono microprogrammi
 - veloce
 - più flessibile della realizzazione HW
- ▷ **SW:** macchina realizzata mediante simulazione, ovvero strutture dati e algoritmi sono a loro volta dei programmi
 - minore velocità
 - maggiore flessibilità

Una macchina (sistema) complessa è suddivisa in livelli di astrazione cooperanti, ma sequenziali e indipendenti.

Ciascun livello è definito da un linguaggio L che è l'insieme delle istruzioni che il livello mette a disposizione per i livelli successivi/superiori.



Tutte le istruzioni ad un livello i sono implementate da programmi a livelli $j < i$.

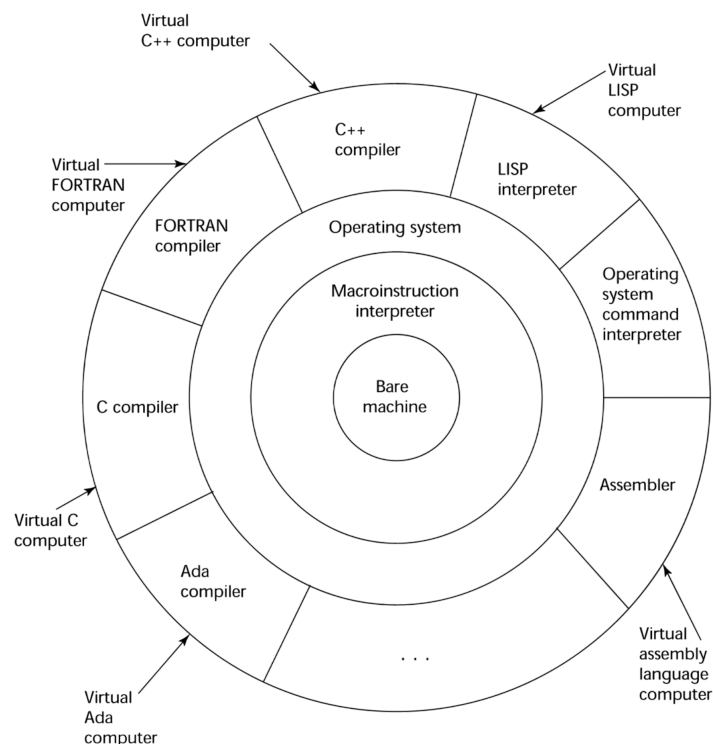


Figura 1.2: livelli di astrazione dei PL

Per Implementare un linguaggio L , dobbiamo realizzare una macchina M_L in grado di eseguirlo. Tali macchine differiscono nel mondo in cui la macchina astratta realizzata e nelle strutture dati utilizzate.

La realizzazione della macchina astratta dipenderà, quindi, da quelle funzionalità che vengono messe a disposizione dei livelli sottostanti.

Realizzare M_L consiste nel realizzare una macchina che traduce L in L_0 , ovvero che interpreta tutte le istruzioni di L come (sequenza di) istruzioni di L_0 .

Si possono distinguere due modalità:

- ◇ **traduzione esplicita** (soluzione compilativa): macchina che traduce programmi scritti in L in programmi scritti in L

$$P_L \xrightarrow{\text{traduzione}} P_{L_0} \xrightarrow{\text{esecuzione}} M_{L_0}$$

- ◇ **traduzione implicita** (soluzione interpretativa): macchina simula ogni istruzione di programmi in L mediante programmi/istruzioni scritti in L_0

$$P_L \xrightarrow{\text{simulazione}} M_{L_0}$$

1.7.1 Soluzione interpretativa

La soluzione interpretativa è una traduzione esplicita, ovvero consiste nella simulazione di L in M_{L_0} .

Notazione

- Prog^L : insieme programmi scritti in L
- D : insieme dati (input e output)
- $P^L \in \text{Prog}^L$ e $\text{in}, \text{out} \in D$
- $\llbracket P^L \rrbracket: D \rightarrow D$ semantica di P^L tale che $\llbracket P^L \rrbracket(\text{in}) = \text{out}$ se P^L eseguito a partire da in restituisce out

Dato P^L su dati D , un interprete $I^{L_0,L}$ è un programma che riceve in input $P^L \in \text{Prog}^L$ e $d \in D$.

Definizione: interprete

Un **interprete** per L , scritto in L_0 , è un programma che realizza la funzione:

$$\underbrace{I^{L_0,L} : (\text{Prog}^L \times D) \rightarrow D}_{I^{L_0,L}(P^L, \text{input}) = P^L(\text{input}) \quad \circ \quad \llbracket I^{L_0,L} \rrbracket(P^L, \text{input}) = \llbracket P^L \rrbracket(\text{input})}$$

$\Rightarrow$ l'interprete applicato a P^L e al dato (input) restituisce esattamente ciò che restituirebbe P^L applicato all'input.

L'interprete, quindi, è un altro programma che prende P^L e lo esegue passo passo.

Questo oggetto (interprete) viene definito **macchina universale**.

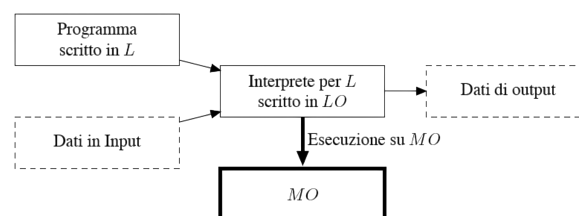


Figura 1.3: schema grafico dell'interprete

1.7.1.1 Operazioni

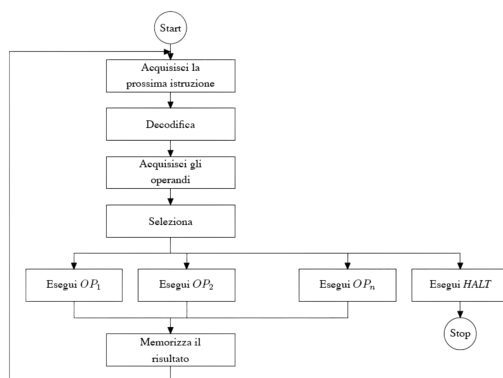
Un interprete è di fatto un ciclo che attraverso una serie di operazioni e funzionalità esegue per simulazione le istruzioni del linguaggio.

Le operazioni principali sono:

- ▷ **elaborazione dei dati primitivi** (dati rappresentati in modo diretto nella memoria)
- ▷ **controllo della sequenza di esecuzione**: gestione del flusso
- ▷ **controllo dei dati**: recupero dati per l'esecuzione delle istruzioni **controllo della memoria**: gestione dell'allocazione dei dati e programmi

1.7.1.2 Struttura

Stabilite le operazioni di cui un interprete ha bisogno, vediamo come opera concretamente. Il ciclo di esecuzione comune a tutti gli interpreti è:



(a) flow-chart

```

begin
  go := true;
  while go do begin
    FETCH(OPCODE, OPINFO) at PC
    DECODE(OPCODE, OPINFO)
    if OPCODE needs ARGS then FETCH(ARGS)
    case OPCODE of
      OP1: EXECUTE(OP1, ARGS)
      ...
      OPn: EXECUTE(OPn, ARGS)
      HLT: go := false;
    if OPCODE has result then STORE(RES);
    PC := PC + SIZE(OPCODE);
  end
end

```

Controllo dati (CD) points to the `if OPCODE needs ARGS` line.

Controllo sequenza (CS) points to the `while go do begin` and `HLT: go := false;` lines.

(b) pseudocodice

1.7.2 Soluzione compilativa

La soluzione compilativa è una traduzione implicita.

Notazione

- Prog^L : insieme programmi scritti in L
- D : insieme dati (input e output)
- $P^L \in \text{Prog}^L$ e $\text{in}, \text{out} \in D$
- $\llbracket P^L \rrbracket: D \rightarrow D$ semantica di P^L tale che $\llbracket P^L \rrbracket(\text{in}) = \text{out}$ se P^L eseguito a partire da in restituisce out

Costruiamo $C^{L_0, L}$ dove L è il linguaggio da implementare (linguaggio sorgente) e L_0 è il linguaggio macchina ospite (linguaggio target).

Definizione: compilatore

Un **compilatore** da L a L_0 è un programma che realizza la funzione:

$$C^{L_0, L}: \text{Prog}^L \rightarrow \text{Prog}^{L_0}, p^L \in \text{Prog}^L$$

$$C^{L_0, L}(P^L) = P^{L_0} \mid \forall \text{in}: P^L(\text{in}) = P^{L_0}(\text{in}) \circ \llbracket C^{L_0, L} \rrbracket(P^L) = P^{L_0} \mid \forall \text{in}: \llbracket P^L \rrbracket(\text{in}) = \llbracket P^{L_0} \rrbracket(\text{in})$$

In questo caso, quindi, la fase di traduzione viene separata dalla fase di esecuzione.

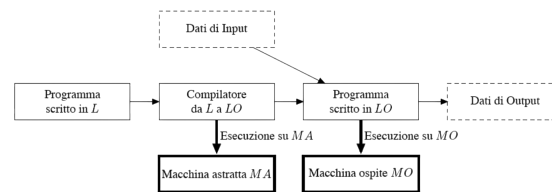


Figura 1.5: schema grafico compilatore

1.7.2.1 Struttura

L'esecuzione di un compilatore passa attraverso varie fasi:

- ▷ **analisi lessicale** (scanner): individua le unità lessicali (variabili, comandi, valori, ecc.)
- ▷ **analisi sintattica** (parser): analizza se gli elementi lessicali sono stati costruiti nel modo corretto mediante l'uso di parse-tree
- ▷ **analisi semantica + generazione codice intermedio**
- ▷ **generazione codice**

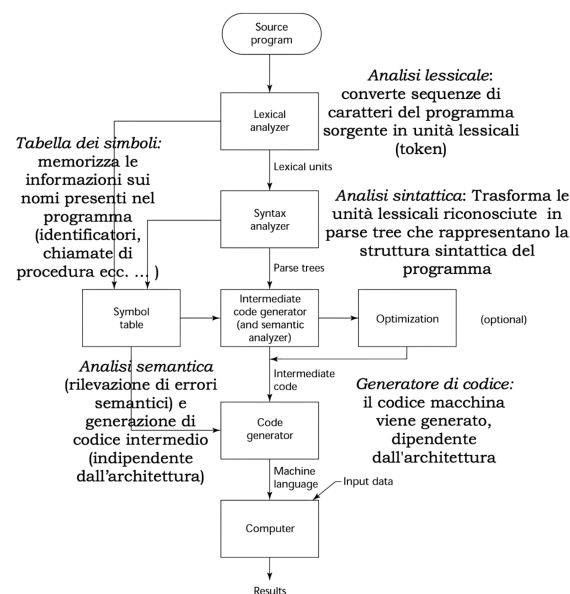
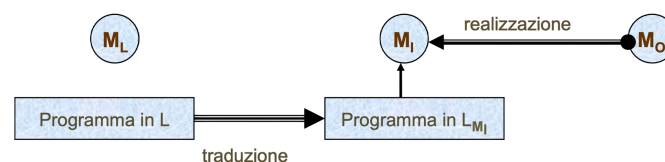


Figura 1.6: flow-chart

1.7.3 Soluzione ibrida

Esiste un compromesso tra compilatore e interprete, ovvero una soluzione ibrida dove il linguaggio ad alto livello viene compilato in un linguaggio a più basso livello che poi viene interpretato.



In questo caso consideriamo il linguaggio L , ad alto livello, per il quale dobbiamo realizzare la macchina astratta M_L .

L viene, quindi, tradotto in un linguaggio intermedio L_{M_i} la cui macchina astratta M_L consiste in un interprete del linguaggio L_{M_i} sulla macchina ospite M_0 .

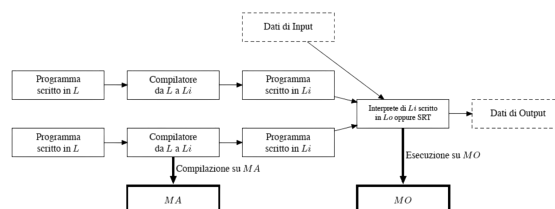


Figura 1.7: schema grafico soluzione ibrida

Esempio: linguaggi di programmazione con soluzione ibrida

Un linguaggio di programmazione che utilizza la soluzione ibrida è Java.

Descrivere i linguaggi di programmazione

Un linguaggio di programmazione è un formalismo artificiale utilizzato per esprimere algoritmi. Anche se artificiale, è sempre un linguaggio e, quindi, deve essere descritto.

Un linguaggio di programmazione non è altro che come un linguaggio naturale, ma semplificato e non ambiguo.

La descrizione di un linguaggio PL avviene tramite:

- ▷ **sintassi** (grammatica): regole di formazione di frasi ben formate (descrive relazione tra segni)
- ▷ **semantica**: attribuzione di significato ai simboli della sintassi (costruisce relazione tra segni e significato)
- ▷ **pragmatica**: modo in cui le frasi del linguaggio di programmazione vengono usate
- ▷ **implementazione**: strumenti per eseguire frasi del linguaggio di programmazione rispettandone la semantica

2.1 Sintassi

Definizione: sintassi

Le **regole sintattiche** del linguaggio specificano quali stringhe di caratteri sono legali nel linguaggio.

La terminologia nella linguistica sono:

- **parola**: sequenza (stringa) di caratteri in un alfabeto
- **frase**: sequenza (ben formata) di parole
- **linguaggio**: insieme di frasi

Nell'ambito dei linguaggi di programmazione, gli stessi nomi prendono nomi differenti:

- ◇ **lessema** (parola): parola (terminale) con significato proprio.
Corrisponde all'**unità sintattica a più basso livello dei PL**.

- ◇ **token** (frase): sequenze di simboli non terminali.
Corrispondono agli **elementi delle categorie sintattiche**.

2.1.1 Descrivere la sintassi

Abbiamo bisogno di strumenti formali per descrivere parole e frasi ben formate di un linguaggio. che permettono, in modo automatico, di verificare l'appartenenza o meno al linguaggio.

Definizione: riconoscitore

Il **riconoscitore** è uno strumento (formale) di riconoscimento che legge in input stringhe sull'alfabeto e decide se la stringa appartiene o meno al linguaggio.

Es. automi a stati finiti (analisi sintattica)

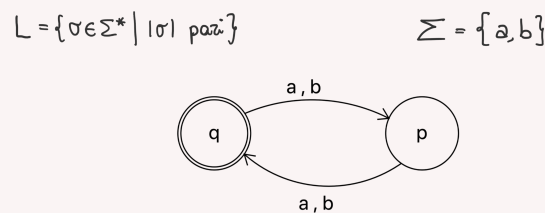


Figura 2.1: esempio di automa a stati finiti

Definizione: generatore

Il **generatore** è uno strumento (formale) che genera stringhe di un linguaggio.

Es. grammatiche CF (parser)

$$S := \varepsilon \mid aaS \mid abS \mid baS \mid bbs$$

Figura 2.2: esempio di stringhe generate da un automa a stati finiti

2.1.2 Grammatiche CF

La grammatica descrive i modi per combinare parole e formare frasi ed è descritta da:

- ▷ **alfabeto**
- ▷ **descrizione lessicale:** descrizione delle parole
- ▷ **descrizione delle frasi**

Esempio: grammatica

$$L = \{\sigma \in \Sigma^* \mid \sigma \text{ palindromo}\}$$

Def: • ε , a e b sono palindromi
 • se σ è palindromo allora lo
 sono anche $a\sigma a$ e $b\sigma b$

$$S := \varepsilon \mid a \mid b \mid aSa \mid bSb$$

Per costruire una grammatica abbiamo bisogno di:

- **vocabolario**
- **regole di composizione**: come combinare le parole (produzioni) per ottenere frasi ben formate

Le grammatiche CF permettono di descrivere la sintassi dei linguaggi di programmazione. Questo generatore di linguaggi è stato sviluppato da Chomsky per descrivere la sintassi dei linguaggi naturali e permette di definire la classe di linguaggi CF.

Definizione: grammatica CF

Una **grammatica CF** è una quadrupla

$$G = \langle V, T, P, S \rangle$$

dove:

- ◊ **V**: insieme dei simboli non terminali
- ◊ **T**: insieme dei simboli terminali ($V \cup T = \emptyset$)
- ◊ **P**: insieme delle produzioni

$$\underbrace{A}_{\in V} \rightarrow \alpha \in (V \cup T)^*$$

a un simbolo non terminale A possiamo sostituire (indipendente dal contesto) la sequenza α di simboli terminali e non

- ◊ $\underbrace{S \in V}$: simbolo iniziale
 rappresenta
 il linguaggio
 da generare

Nella grammatica i simboli terminali costituiscono il vocabolario (collezione di parole/token), invece, i simboli non terminali sono le categorie sintattiche (espressioni, comandi, dichiarazioni, ecc.).

Le produzioni sono le regole di formazione/composizione (istruzioni . . . programmi).

Il linguaggio è l'insieme di tutte le frasi (stringhe di simboli terminali) generate a partire dal simbolo iniziale S.

2.1.2.1 Notazione BNF

La notazione BNF viene ideata nel 1959 da Bakus per Algol60.

Questa notazione è usata per descrivere grammatiche CF, dove si usano intere parole come simboli terminali, mentre i non terminali sono identificati racchiudendoli tra parentesi angolate <>.

Per le produzioni, invece, viene utilizzato il simbolo := al posto della freccia ->.

$$\begin{aligned} \langle A \rangle &::= \alpha \langle B \rangle \gamma \mid \alpha \\ \langle B \rangle &::= \varepsilon \mid \beta_1 \mid \beta_2 \end{aligned}$$

Figura 2.3: esempio di notazione BNF

2.1.3 Descrivere un semplice linguaggio imperativo

Un linguaggio può essere descritto anche in modo informale descrivendo quali caratteristiche deve avere qual è il loro significato:

- ▷ niente dichiarazioni
- ▷ solo variabili ed espressioni booleane
- ▷ assegnamento e composizione sequenziale
- ▷ comando condizionale
- ▷ comando imperativo (loop)

Anche i linguaggi di programmazione sono descritti a livelli di astrazione, ognuno descritto da una grammatica.

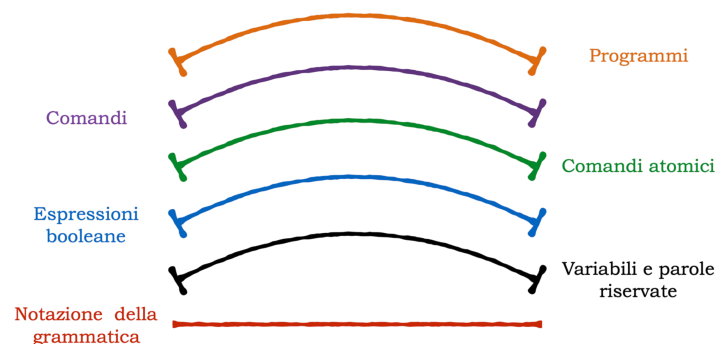


Figura 2.4: livelli di astrazione dei PL

Notazione grammatica

```
<bExpr> B -> true | false | v | (not B) | (B and B) | (B or B)

<atomic com> A -> v := B

<com> C -> A | C ; C | if B then C else C | while B do C

<program> S -> C
```

2.1.3.1 Analisi semantica

Esistono dei vincoli che la grammatica CF non può catturare:

- ▷ **vincoli contestuali**: dichiarare una variabile prima dell'uso, n° di parametri di una chiamata

Il problema è che non esistono algoritmi per il riconoscimento di stringhe generate, quindi, non ci sono algoritmi per fare il parser di queste stringhe.

Per questo la scelta ricade su una specifica del linguaggio mediante grammatica CF (sintassi) e poi, come parte della semantica (semantica statica) vengono specificati i vincoli contestuali.

2.2 Semantica**2.3 Categorie sintattiche**

Comandi

I **comandi** sono la categoria sintattica che permettono di rappresentare trasformazioni irreversibili dello stato.

L'astrazione dello stato della macchina è rappresentata dalla memoria.

Definizione: memoria

La **memoria** è l'insieme delle associazioni tra locazioni e valori (es. array illimitato: indice = locazione, valore = contenuto della locazione).

La trasformazione dello stato, avviene tramite un meccanismo definito **assegnamento**.

Definizione: assegnamento

L'**assegnamento** è un comando che permette di modificare il contenuto della memoria.

Ne deriva che non è pensabile che si possa utilizzare direttamente la locazione di memoria. Le locazioni vengono riferite attraverso identificatori chiamate variabili.

Definizione: variabili

Le **variabili** sono identificatori associati a locazioni, che a loro volta, nella memoria, sono associate a valori che possono cambiare durante l'esecuzione.

Per il momento, quindi, supponiamo che le variabili siano nomi astratti per le celle di e supponiamo di associare direttamente il valore (modificabile) a una variabile.

Vediamo alcuni aspetti dell'uso delle variabili che rendono questo modello poco adatto:

- ▷ quando ci riferiamo a un valore modificabile mediante una variabile, lo possiamo fare sia per accedere al suo valore in lettura, sia per modificare il suo valore
- ▷ per riferirsi a oggetti da modificare potremmo utilizzare funzioni della variabile e non il semplice nome (es. array $a[x+1]$)
- ▷ gli identificatori possono essere ridefiniti causando modifiche reversibili all'ambiente. Ciò significa che per un certo intervallo di tempo, una variabile potrebbe avere in altro significato, mentre il valore che aveva prima della ridefinizione non deve andare perso perchè potrebbe ritornare valido (es. uscita da una procedura)

Introduciamo, quindi, la locazione come astrazione della memoria fisica e la variabile come astrazione della locazione.

Significa che introduciamo qualcosa di intermedio tra il nome della variabile e il nome riferito, che modella la memoria nella sua visione ideale (illimitata).

In questo modo è possibile che una variabile possa avere indirizzi differenti in momenti diversi dall'esecuzione, oppure che una variabile possa avere differenti indentificatori in punti diversi del programma.

Si distinguono l'associazione tra nome e locazione che non cambia in modo reversibile durante l'esecuzione e l'associazione tra locazione e valore che cambia in modo irreversibile durante l'esecuzione.

Definizione: locazione

La **locazione** è il contenitore di valori che possono essere inizializzati, indefiniti o definiti.

Definizione: memoria

La **memoria** è la collocazione delle associazioni tra locazioni e valori contenuti, che possono essere costantemente cambiati.

Questo significa che la locazione viene creata da una dichiarazione.

Inoltre, la locazione ha un tempo di vita determinato dallo scope della dichiarazione.

Definizione: identificatore costante

un **identificatore costante** è un identificatore che può assumere un valore predefinito e non è ulteriormente modificabile.

Definizione: identificatore variabile

Un **identificatore variabile** è un identificatore il cui valore può essere aggiornato.